



The silver line

Microsoft's web application framework has transformed into a full-fledged application framework with its new avatar and is slowly gaining ground in a world nearly dominated by Flash

Rahil Banthia

readersletters@thinkdigit.com

Silverlight, although not technically a part of the .Net Framework, was updated with the recent release of .Net 4. It has come far from being a small subset of the Framework's core features running as a plugin for web browsers to a full-fledged cross-platform application framework like Adobe Flash. The WPF graphics model of Silverlight, leveraging existing .Net developers bundled with out-of-browser capabilities and plugins for nearly every web browser including mobile platforms pose an ever increasing threat to the dominant Flash. Let us delve deeper into Silverlight 4 and see what new features it brings to the table and if they are really worth all the hype.

Silverlight 4 includes many new features such as web camera and microphone support, right-click event handling, improved rich text boxes and

data binding features, modular development with MEF, and support for several new languages. All of this, if rightly implemented will enhance the building of business and media applications using this framework, even applications that reach beyond the browser.

Right-click event handling

What this essentially translates into, is that right-mouse-click events can now display a custom context menu instead of the default Silverlight menu, or invoke direct commands that could be used in games. The events are routed to a particular element and use the same MouseButtonEventArgs that the MouseLeftButtonUp and MouseLeftButtonDown events use so you can determine which element was affected and where on that element a right mouse button event occurred in order to show a context menu at that location.

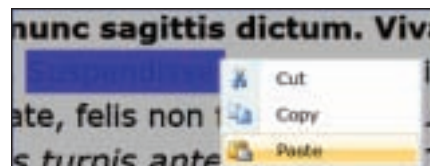
When the MouseRightButtonDown event is handled and the MouseBut-

tonEventArgs.Handled property is set to true, the default Silverlight configuration menu will not be displayed. The MouseButtonEventArgs.StylusDevice.DeviceType property will report the source device type for the event such as mouse, stylus, or touch. The following code is required to implement the right click functionality and display a custom menu control:

```
public MainPage()
{
    InitializeComponent();
    mediaElement.MouseRightButtonDown +=
        new MouseButtonEventHandler(MouseRightButtonDownHandler);
    mediaElement.MouseRightButtonUp +=
        new MouseButtonEventHandler(MouseRightButtonUpHandler);
}

private void MouseRightButtonDownHandler(object sender, MouseButtonEventArgs e)
{
    e.Handled = true;
}

private void MouseRightButtonUpHandler(object sender, MouseButtonEventArgs e)
{
    var menu = new MyContextMenu(mediaElement);
    menu.Show(e.GetPosition(LayoutRoot));
}
```



This is how the right-click contextual menu will look